

COS 433: Cryptography

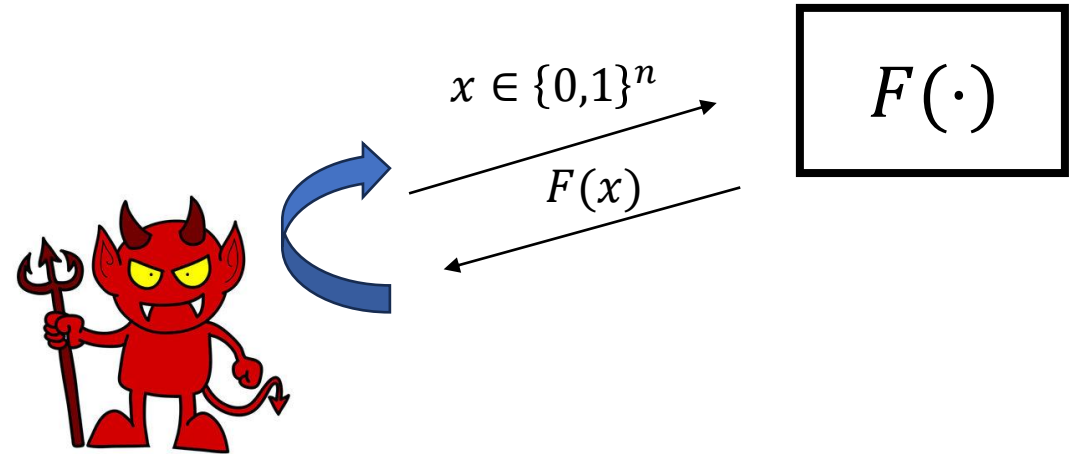
Reminders:

- Problem Set 2 due 10/7

Last time: PRFs

Pseudorandom Functions (PRFs)

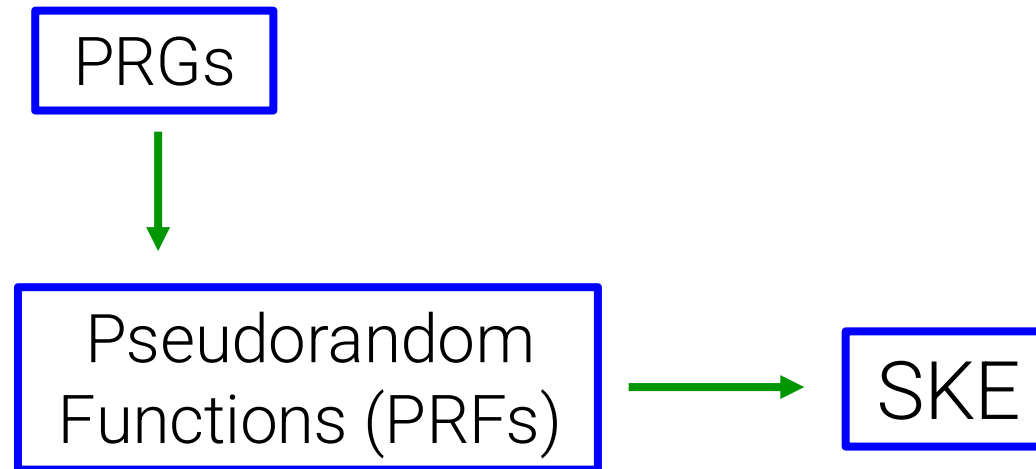
A poly-time **oracle algorithm** $A^{F(\cdot)}(1^\lambda)$ is an algorithm that can call a function $F(\cdot)$ on inputs of length $\text{poly}(\lambda)$ **at unit cost**



Definition: An efficiently computable function family $\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

$$\text{For all polynomial-time } A^{F(\cdot)}, A^{\text{PRF}(\textcolor{red}{s}, \cdot)}(1^\lambda) \approx_c A^{\textcolor{green}{F}(\cdot)}(1^\lambda)$$

Summary



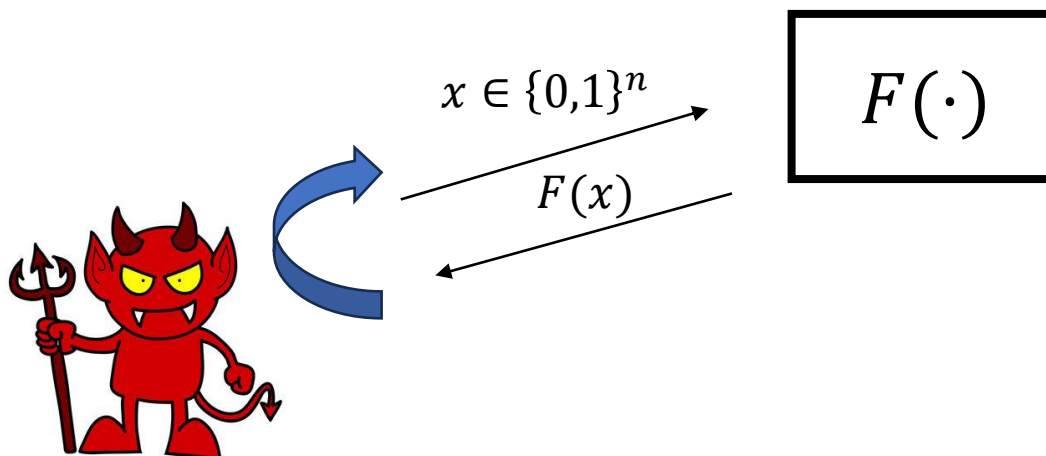
Next: what do “practical” SKE schemes look like?

Pseudorandom Permutations/Block Ciphers

Pseudorandom Permutations (PRPs)

Definition: An efficiently computable function family $P: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^n$ is a PRP family if:

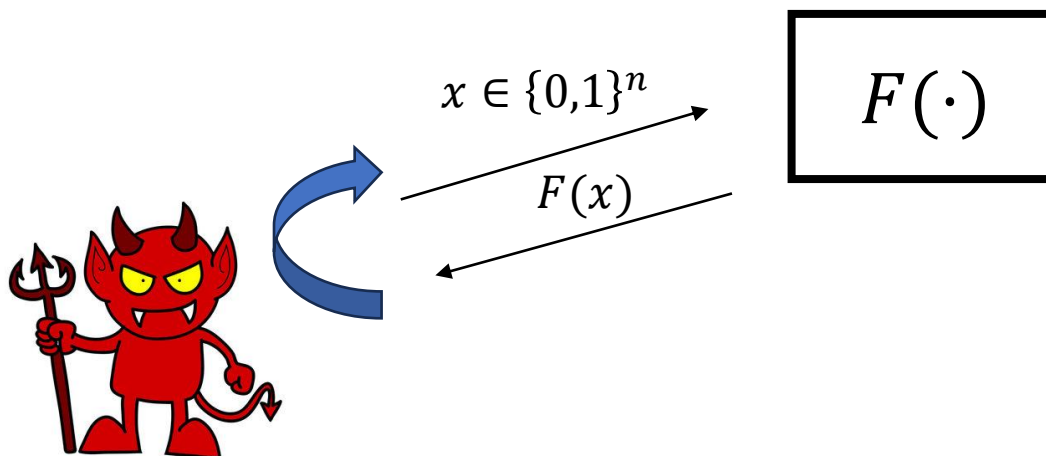
1. **Invertibility:** there is an efficiently computable inverse map $P^{-1}(s, \cdot)$ such that $P^{-1}(s, P(s, x)) = x$ for all x .
2. For all polynomial-time $A^{F(\cdot)}$, $A^{P(s, \cdot)}(1^\lambda) \approx_c A^{P(\cdot)}(1^\lambda)$



Pseudorandom Permutations (PRPs)

Definition 2: An efficiently computable function family $P: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^n$ is a PRP family if:

1. **Invertibility:** there is an efficiently computable inverse map $P^{-1}(s, \cdot)$ such that $P^{-1}(s, P(s, x)) = x$ for all x .
2. For all polynomial-time $A^{F(\cdot)}$, $A^{P(s, \cdot)}(1^\lambda) \approx_c A^{F(\cdot)}(1^\lambda)$ (random **function**)

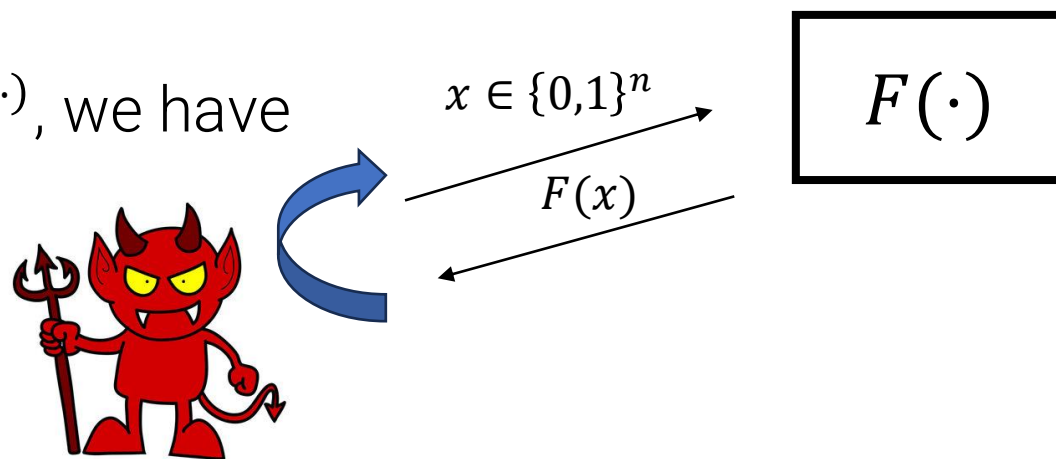


Pseudorandom Permutations (PRPs)

Theorem: Definition 1 is equivalent to Definition 2

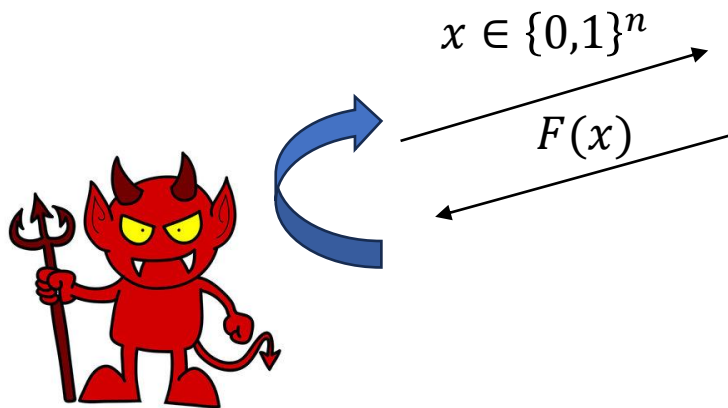
Proof: the claim is that for all polynomial-time $A^{F(\cdot)}$, we have

$$A^{P(\cdot)}(1^\lambda) \approx_c A^{F(\cdot)}(1^\lambda)$$



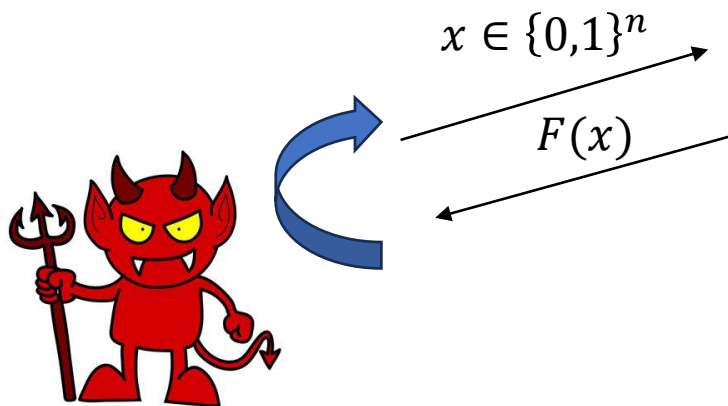
In fact: claim is true even **for inefficient** $A^{F(\cdot)}$, as long as A only calls $F(\cdot)$ **polynomially many times**! (“Query-bounded” adversaries)

Intuition: if you query $F(\cdot)$ poly many times, highly unlikely to encounter collision



$F(\cdot)$:

- Maintain a list of (x, y)
- If query is on the list, output y
- If not, sample y and add (x, y) to list

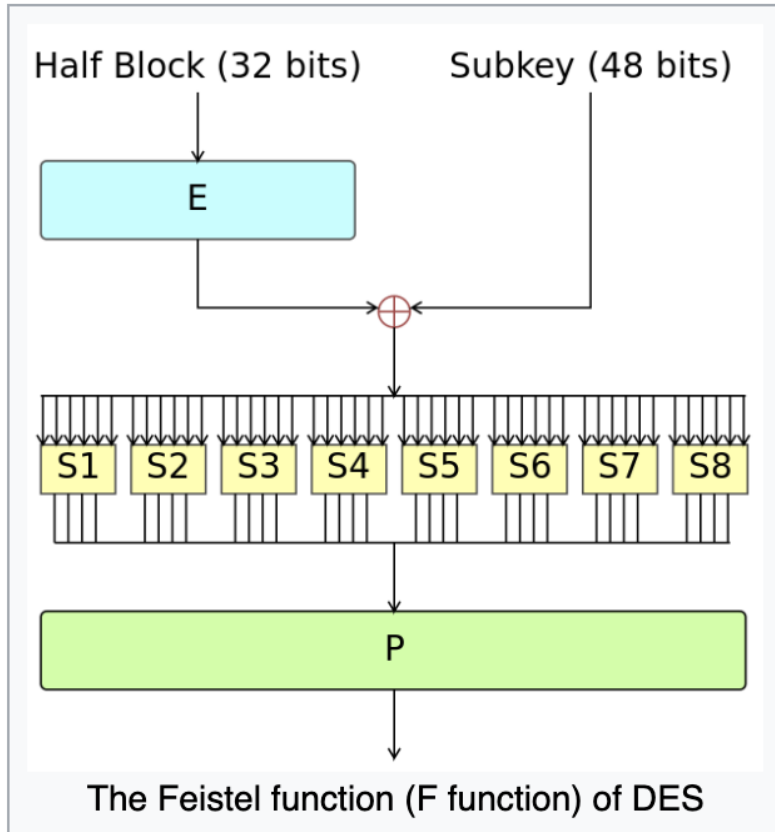


$P(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not, sample $y \notin S$ and add (x, y) to list

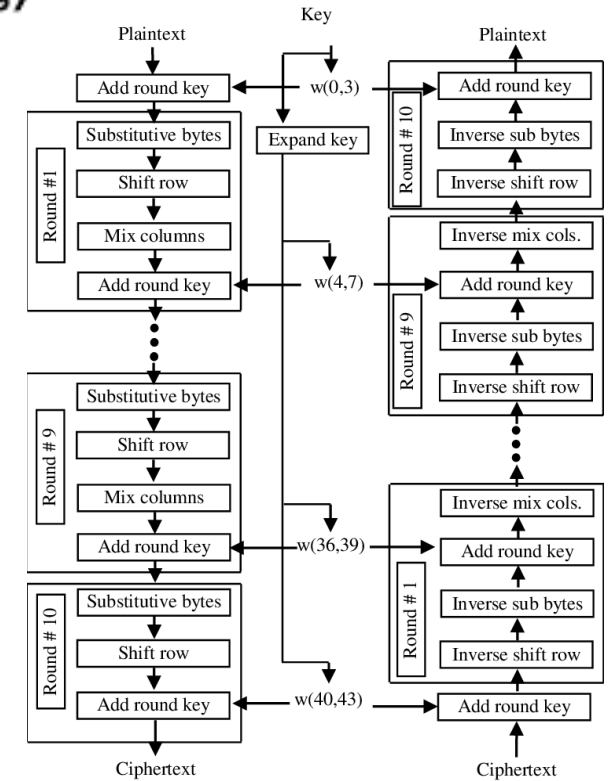
Why do we care about PRPs?

Data Encryption Standard



Real-world SKE
implementations

“Block ciphers”



Encryption from PRPs

Given a PRP, **several ways** to implement an encryption scheme (“modes of operation”). All involve breaking the message into “**blocks**” of length n . Low communication overhead!

- **ECB Mode:**

$$\text{Enc}(k, m_1, \dots, m_t) = P(k, m_1), P(k, m_2), \dots, P(k, m_t)$$

$$\text{Dec}(k, c_1, \dots, c_t) = P^{-1}(k, c_1), P^{-1}(k, c_2), \dots, P^{-1}(k, c_t)$$

Pros: very simple and length-preserving! **Cons:** deterministic, not CPA-secure

- **CBC Mode:**

$$\text{Enc}(k, m_1, \dots, m_t) = (\text{IV}, c_1 = P(k, \text{IV} \oplus m_1), c_2 = P(k, c_1 \oplus m_2), \dots, P(k, c_{t-1} \oplus m_t))$$

Pros: CPA-secure. **Cons:** highly sequential Enc (though Dec parallel)

- **CTR Mode:**

$$\text{Enc}(k, m_1, \dots, m_t) = (\text{IV}, P(k, \text{IV}) \oplus m_1, P(k, \text{IV} + 1) \oplus m_2, \dots, P(k, \text{IV} + t - 1) \oplus m_t)$$

Pros: CPA-secure and parallel! **Cons:** can't decrypt if you lose a block

Encryption from PRPs

Takeaways:

- **PRPs** are versatile; an invertible permutation is a useful off-the-shelf tool
- Informal intuition: **naturally-occurring permutations** are “more secure” (in terms of **concrete (T, ϵ)**) than functions with collisions.
- Important design paradigms naturally yield PRPs as a “base form” of cryptography, rather than a PRG or other kind of PRF.

Constructing PRPs

Constructing PRPs

1. Theorem (Luby-Rackoff): if PRFs exist, then PRPs exist!
 - “Feistel Networks”
 - Related to the (heuristic) construction of DES.
2. Substitution-Permutation Networks (SPNs)
 - (heuristic) construction of AES.

Constructing PRPs

1. Theorem (Luby-Rackoff): if PRFs exist, then PRPs exist!
 - “Feistel Networks”
 - Related to the (heuristic) construction of DES.

Constructing PRPs

Idea: “reversible computation”

$$\begin{array}{c} (x, y) \in \{0,1\}^{2n} \\ \downarrow \\ P_k \\ (x_1, y) = (x \oplus F_k(y), y) \end{array}$$

Is this secure?

Constructing PRPs

Idea: “iterated reversible computation”

$$(x, y) \in \{0,1\}^{2n}$$



$$P_{k_1, k_2} \quad (x_1, y) = (x \oplus F_{k_1}(y), y)$$



$$(x_1, y_2) = (x_1, y \oplus F_{k_2}(x_1))$$

Is this secure?

$$P_{k_1, k_2}(x, y) \oplus P_{k_1, k_2}(x', y) = (x \oplus x', \text{stuff})$$

Constructing PRPs

Idea: “iterated reversible computation”

$$(x, y) \in \{0,1\}^{2n}$$



$$P_{k_1, k_2, k_3} \quad (x_1, y) = (x \oplus F_{k_1}(y), y)$$



$$(x_1, y_2) = (x_1, y \oplus F_{k_2}(x_1))$$



$$(x_3, y_2) = (x_1 \oplus F_{k_3}(y_2), y_2)$$

Is this secure?

Constructing PRPs

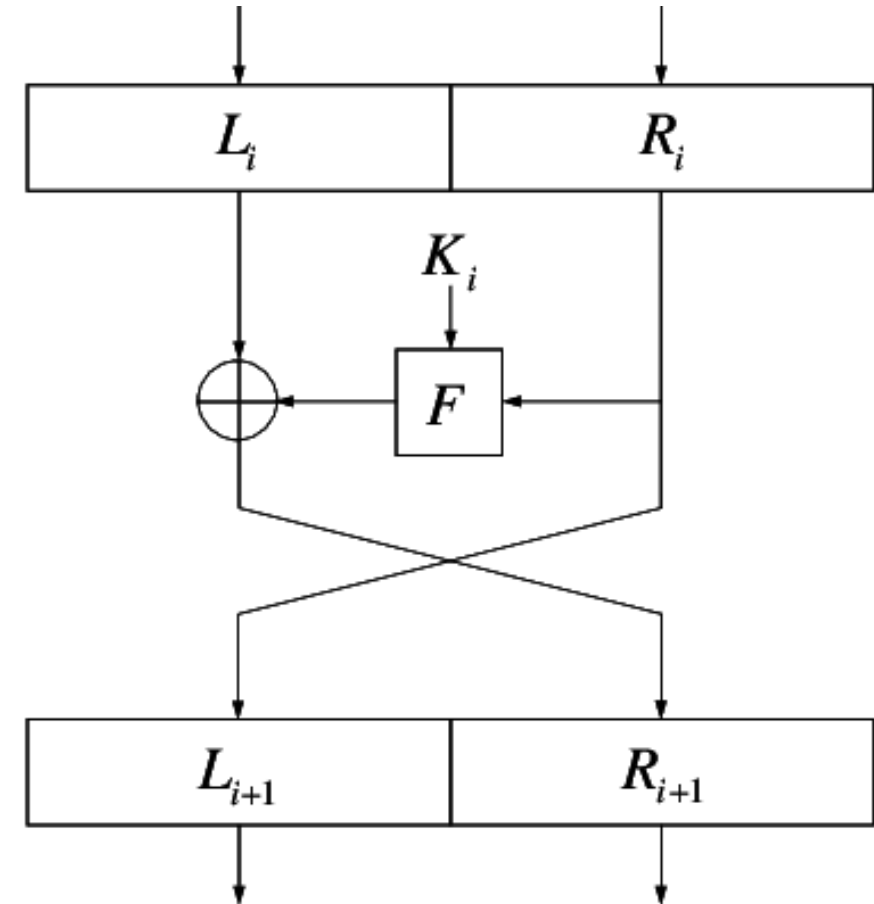
Idea: “iterated reversible computation”

$$\begin{array}{c} (x, y) \in \{0,1\}^{2n} \\ \downarrow \\ P_{k_1, k_2, k_3} \quad (x_1, y) = (x \oplus F_{k_1}(y), y) \\ \downarrow \\ (x_1, y_2) = (x_1, y \oplus F_{k_2}(x_1)) \\ \downarrow \\ (x_3, y_2) = (x_1 \oplus F_{k_3}(y_2), y_2) \end{array}$$

Theorem [Luby-Rackoff]: if $F(\cdot, \cdot)$ is a PRF and k_1, k_2, k_3 are sampled i.i.d., then P_{k_1, k_2, k_3} is a PRP!

Feistel Networks

- r rounds, "round keys" $k_1, \dots, k_r$
- F : "round function"
- Each round: F_{k_i} -gate, swap input halves.
- Luby-Rackoff:
 - $r = 3$
 - $F(\cdot, \cdot)$ = a PRF
 - i.i.d. round keys

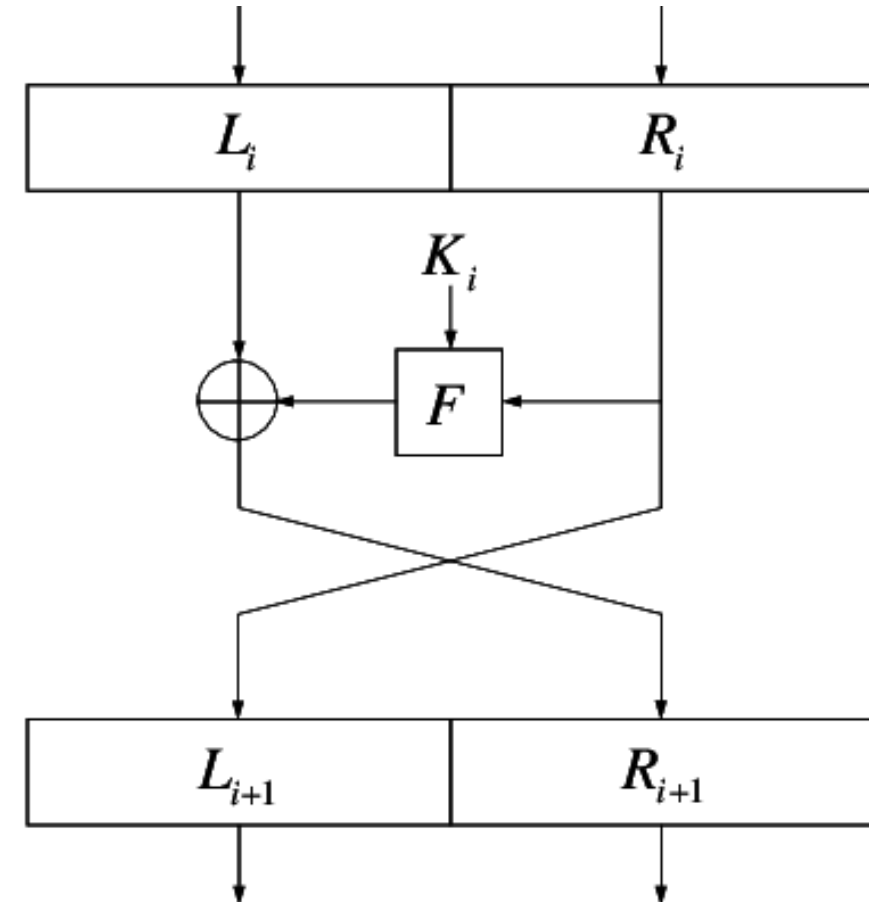
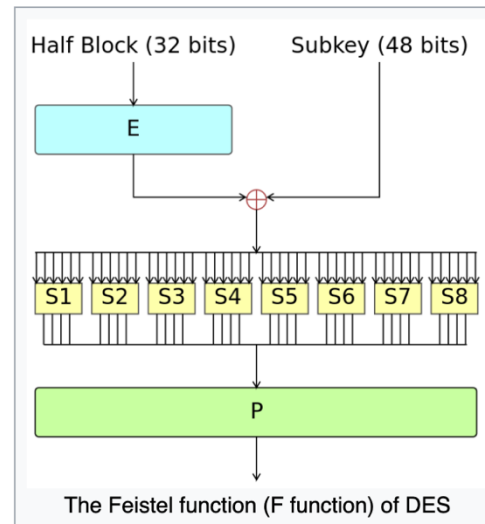


Feistel Networks

- Two “sources” of security:
 - Security of each F_{k_i}
 - Number of rounds r

- DES block cipher:

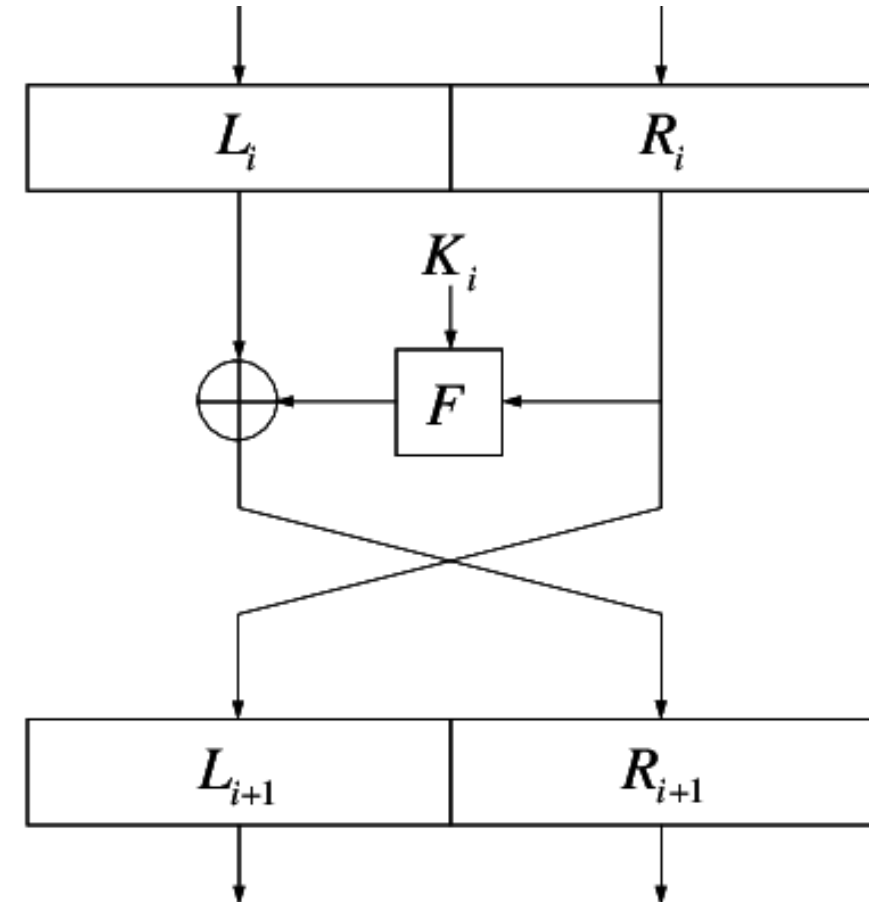
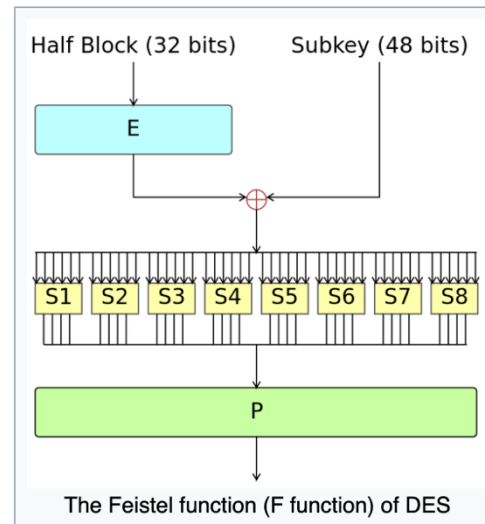
- $r = 16$
- $F_k =$
- round keys are **correlated** in order to minimize key length



Feistel Networks

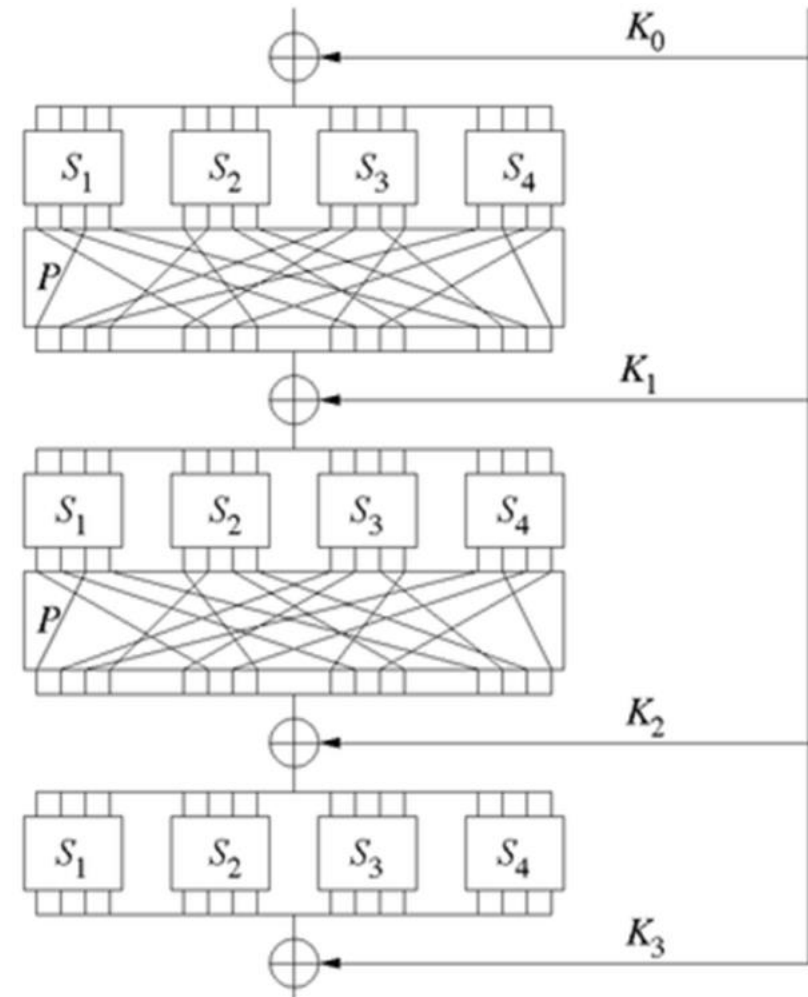
Note: DES keys too short (56 bits) to be secure in modern computing era

- DES block cipher:
 - $r = 16$
 - $F_k =$
 - round keys are **correlated** in order to minimize key length



Substitution-Permutation Networks (AES)

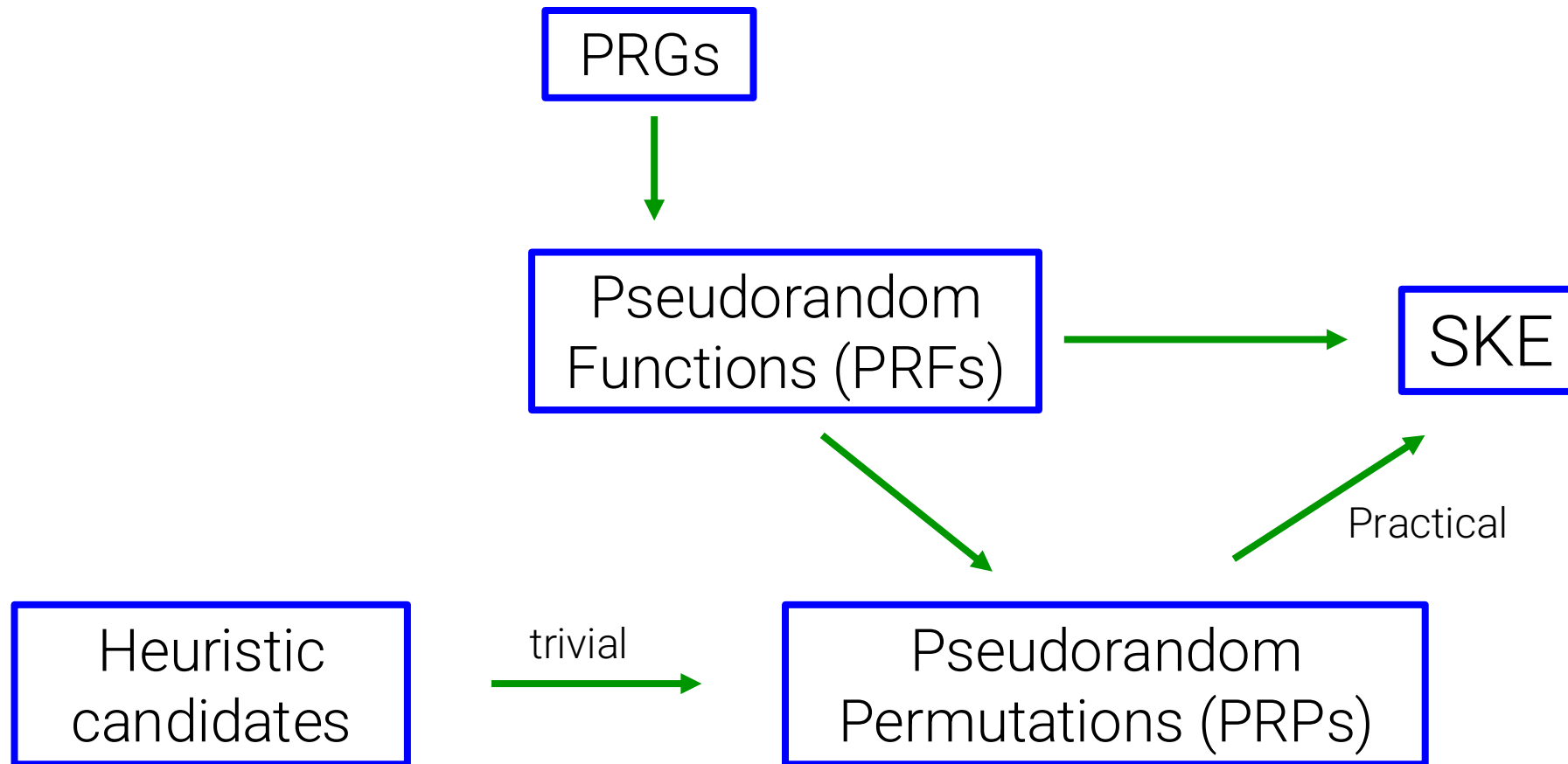
- Similar to DES blueprint
- r rounds, round keys k_i (AES: $r = 10$)
- Round function is a permutation to start with:
 - S -box: highly non-linear **substitution** on sub-blocks
 - Linear **permutation** P on entire domain



Hope (no proof): each round scrambles more, eventually pseudorandom.

See KL Section 7.2 for more details

Summary



Next: Authentication and CCA-security

What if

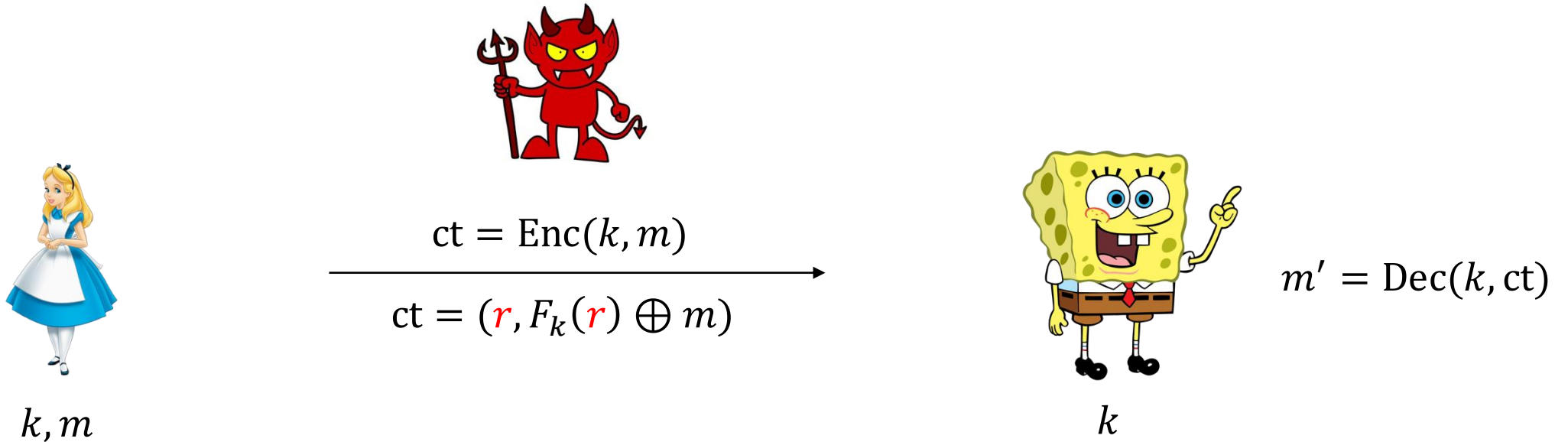


impersonates



?

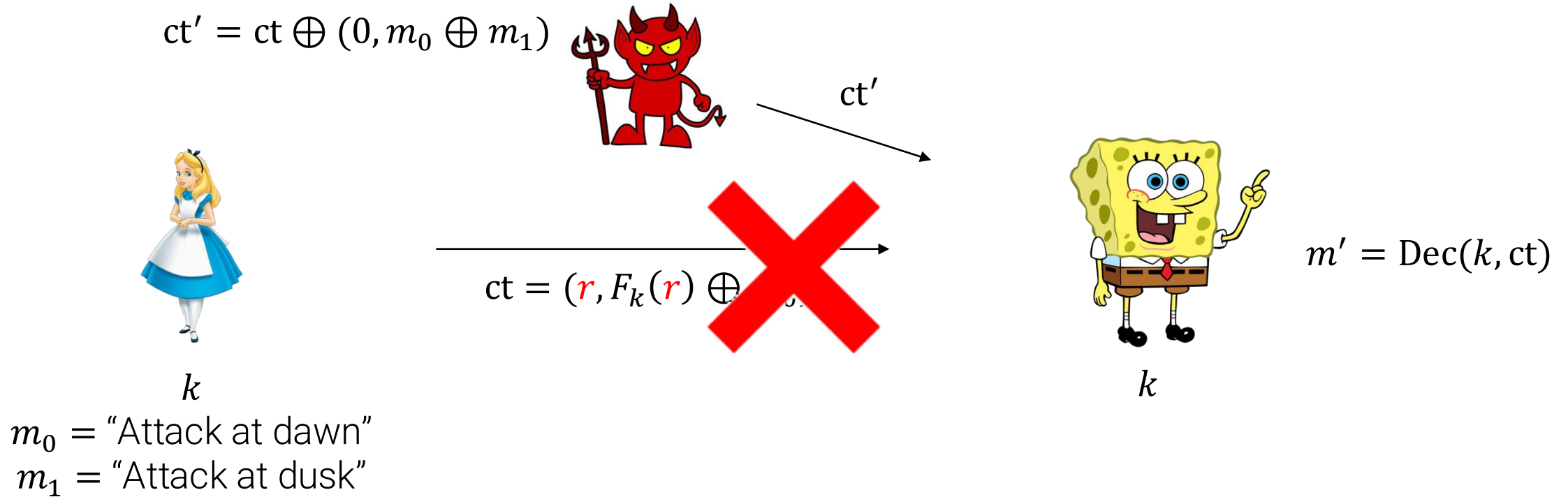
Secret-Key Encryption



We know: this scheme is CPA-secure if F is a PRF family.

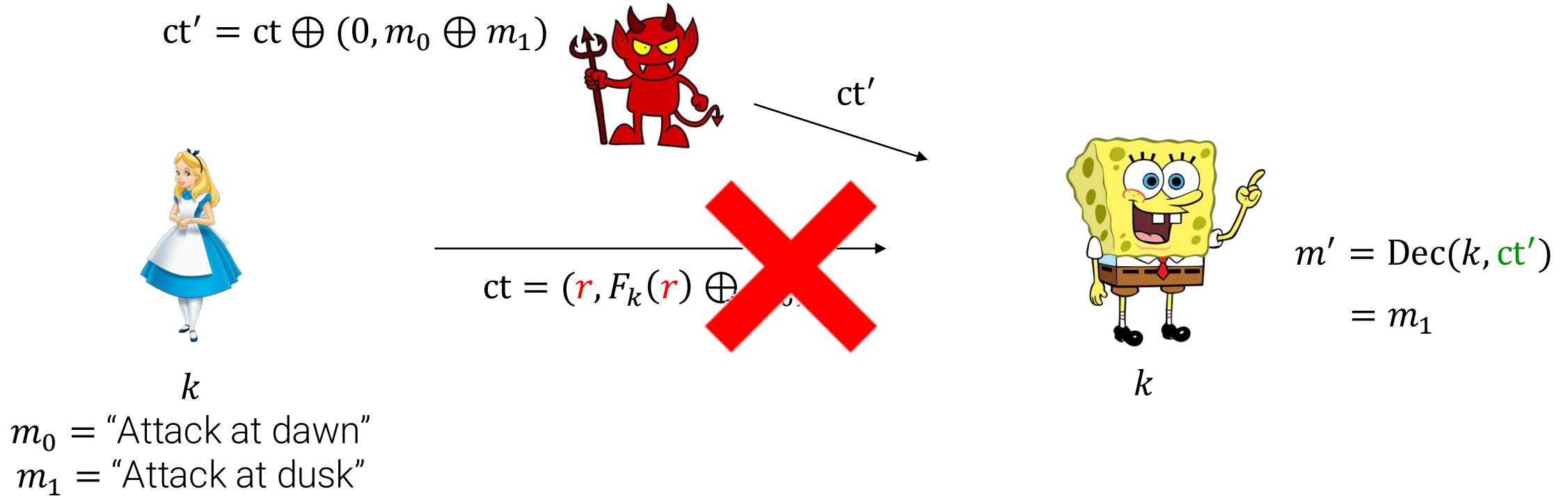
Is this secure enough to use in real life?

Man in the Middle (MITM) Attacks



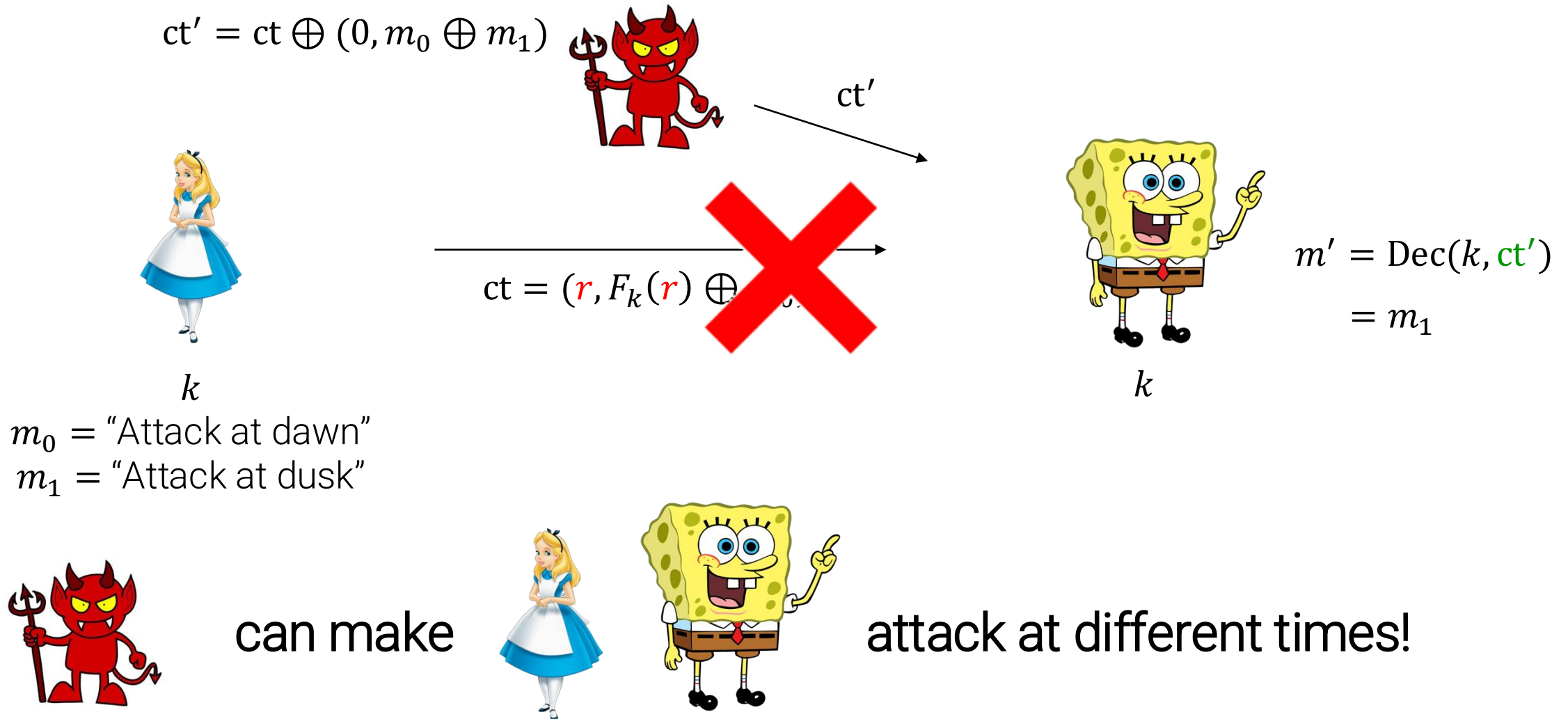
Is this secure enough to use in real life?

Man in the Middle (MITM) Attacks

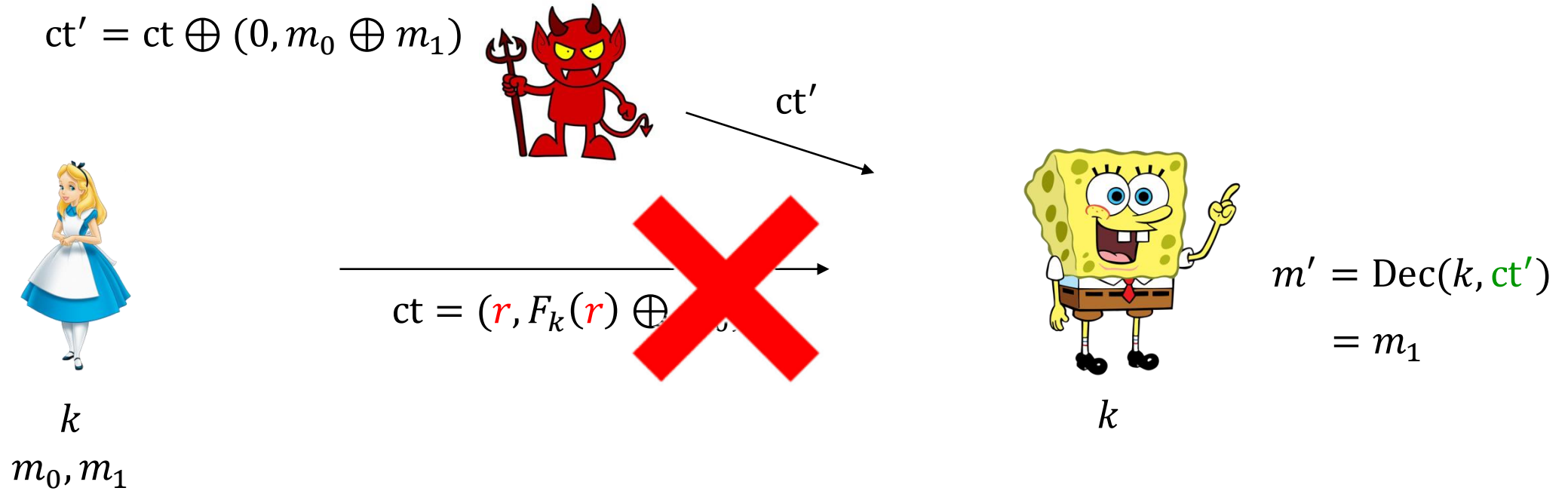


Is this secure enough to use in real life?

Man in the Middle (MITM) Attacks

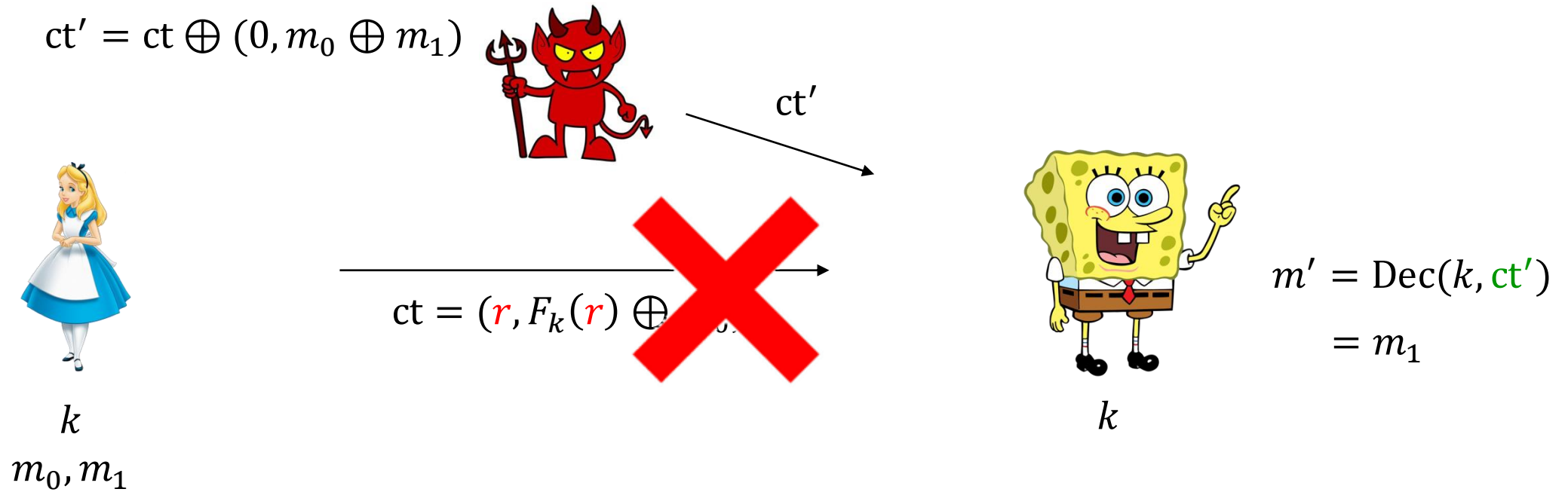


Man in the Middle (MITM) Attacks



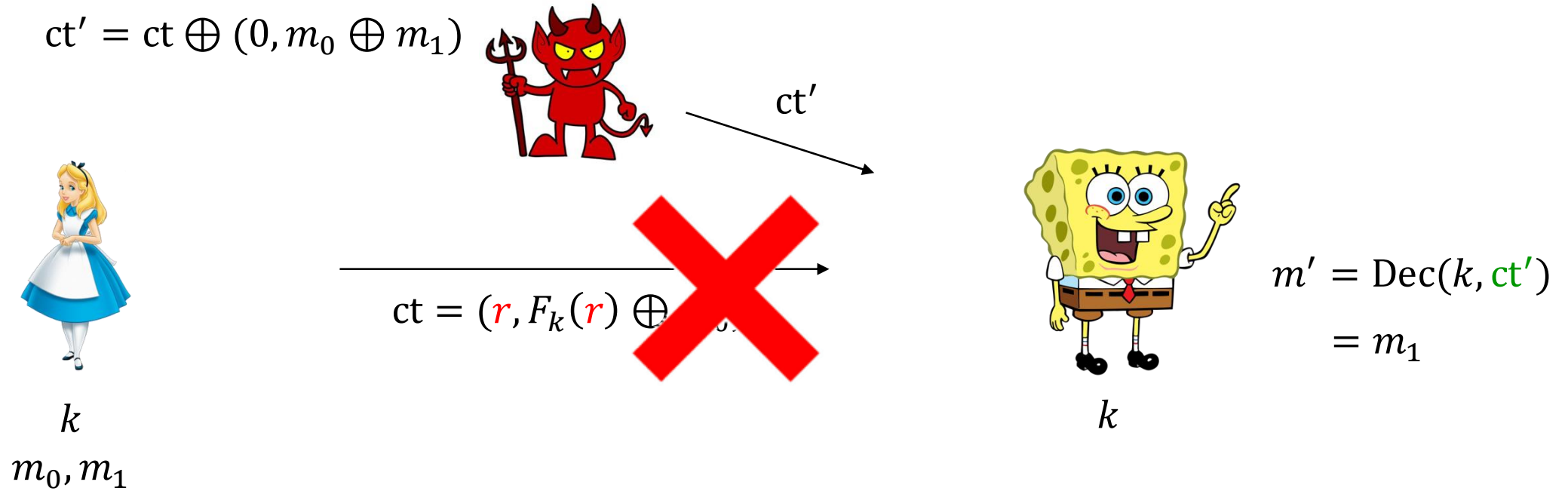
- **Problem:** how does  know that he is receiving a message from  ?
- We need a way to enforce **integrity** of the ciphertexts.

Man in the Middle (MITM) Attacks



Philosophical question: is this scheme insecure?

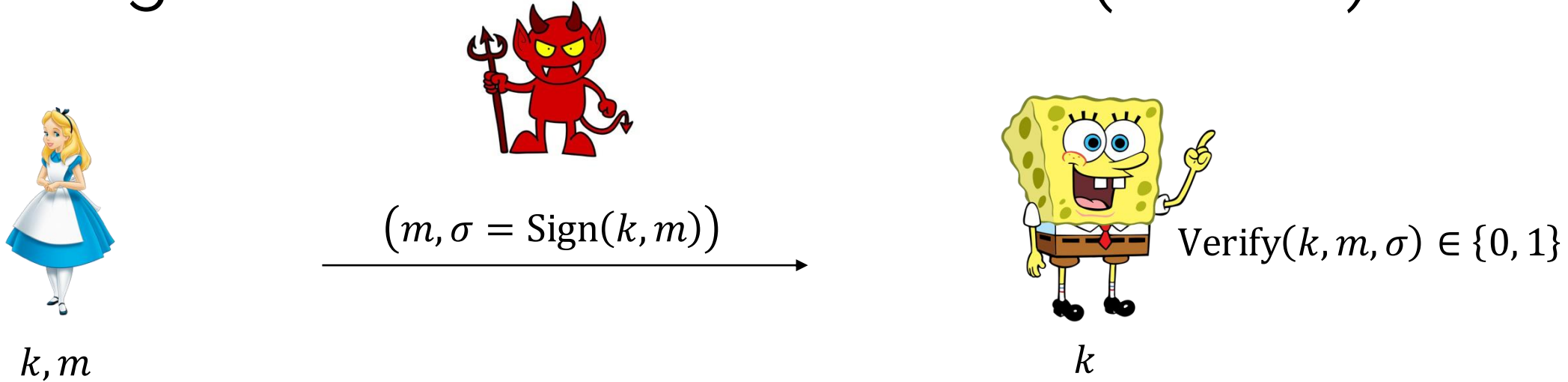
Man in the Middle (MITM) Attacks



- **Answer #1:** No! It's CPA-secure! Integrity is an issue orthogonal to message privacy, and we shouldn't conflate them.
- **Answer #2:** Yes! **There is a way to view this as breaking a form of secrecy!**

Step 1: treat integrity as a standalone problem

Message Authentication Codes (MACs)

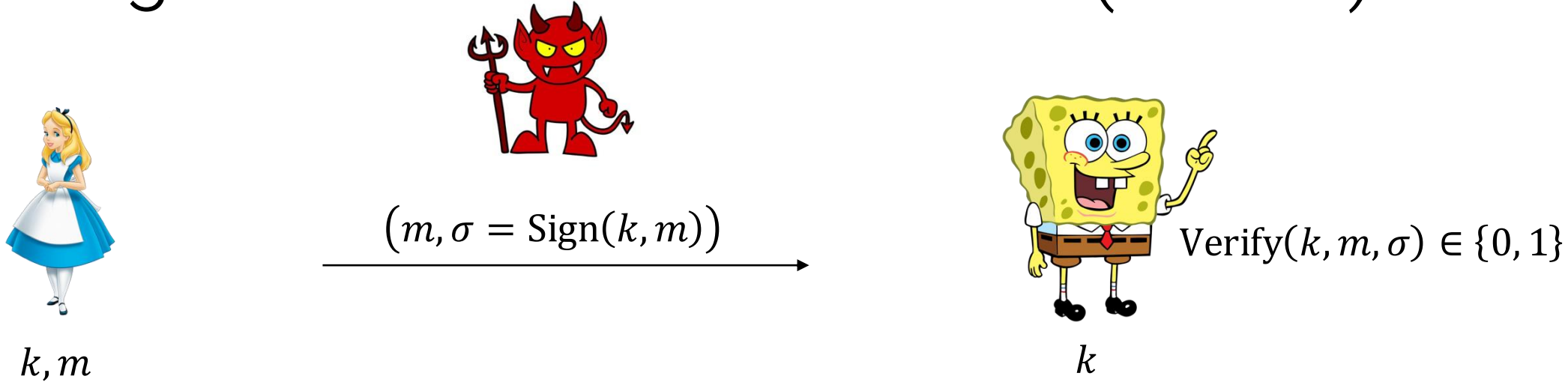


Definition: A **message authentication code (MAC)** scheme with message length $n = n(\lambda)$ consists of efficient algorithms **Sign**, **Verify**. It must satisfy the following properties.

- **Correctness:** for all k, m , $\text{Verify}(k, m, \text{Sign}(k, m)) = 1$ with probability 1.
- **Unforgeability:** “an efficient MITM should not be able to send a fake message”

???

Message Authentication Codes (MACs)

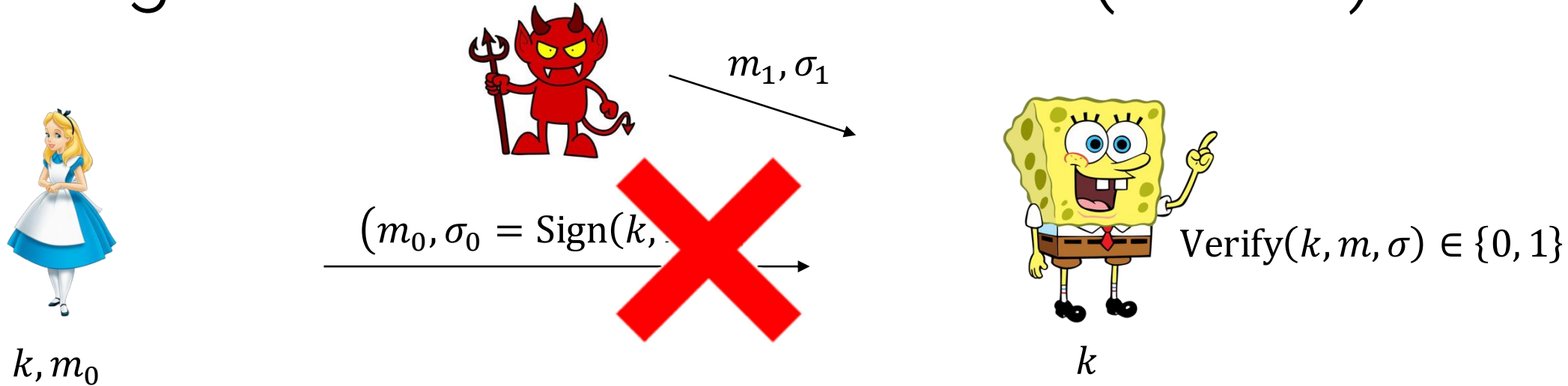


Definition: A **message authentication code (MAC)** scheme with message length $n = n(\lambda)$ consists of efficient algorithms **Sign**, **Verify**. It must satisfy the following properties.

- **Correctness:** for all k, m , $\text{Verify}(k, m, \text{Sign}(k, m)) = 1$ with probability 1.
- **Unforgeability (attempt):** for all efficient $A(m, \sigma) \rightarrow (m', \sigma')$ and all $m_0 \neq m_1$:

$$\Pr[A(m_0, \text{Sign}(k, m_0)) = (m_1, \sigma_1) : \text{Verify}(k, m_1, \sigma_1) = 1] = \text{negl}(\lambda)$$

Message Authentication Codes (MACs)

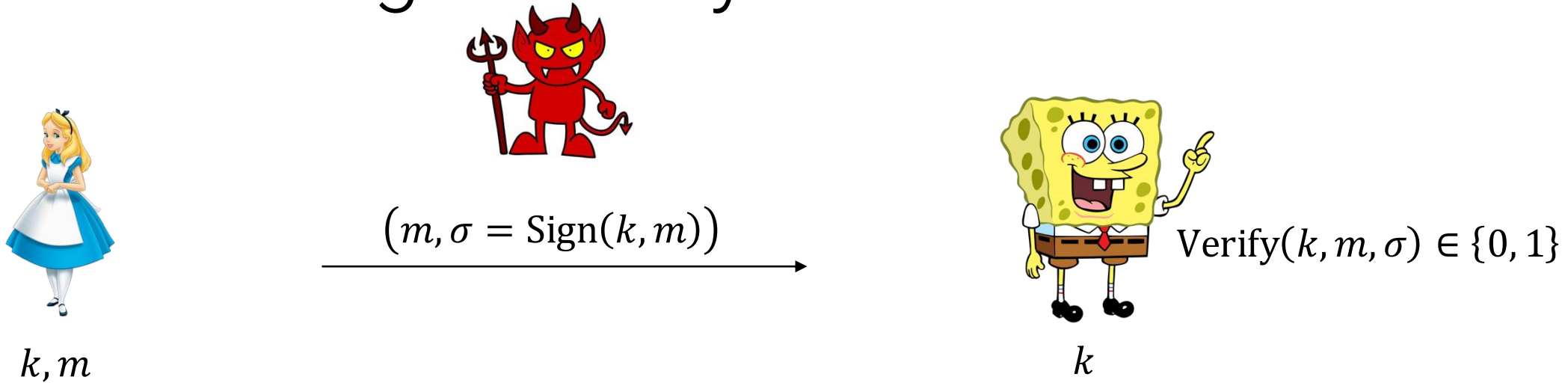


Definition: A **message authentication code (MAC)** scheme with message length $n = n(\lambda)$ consists of efficient algorithms Sign , Verify . It must satisfy the following properties.

- **Correctness:** for all k, m , $\text{Verify}(k, m, \text{Sign}(k, m)) = 1$ with probability 1.
- **Unforgeability (attempt):** for all efficient $A(m, \sigma) \rightarrow (m', \sigma')$ and all $m_0 \neq m_1$:

$$\Pr[A(m_0, \text{Sign}(k, m_0)) = (m_1, \sigma_1) : \text{Verify}(k, m_1, \sigma_1) = 1] = \text{negl}(\lambda)$$

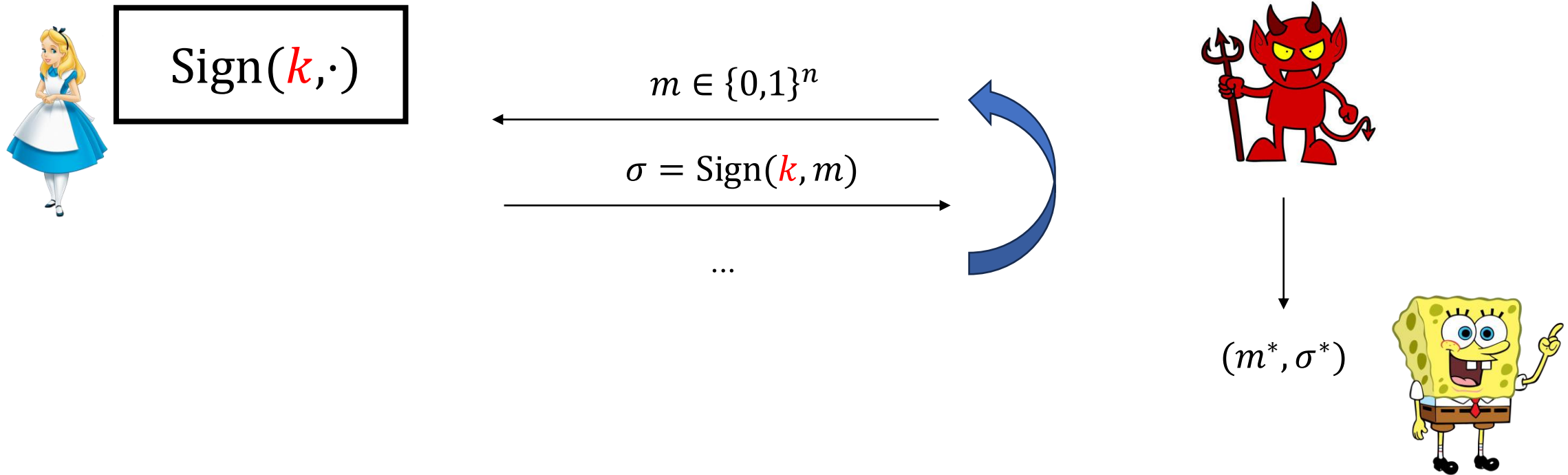
Defining Security for MACs



Attack Model:

-  can intercept messages (and prevent their delivery).
-  wants to trick  into accepting a message that  did not send.

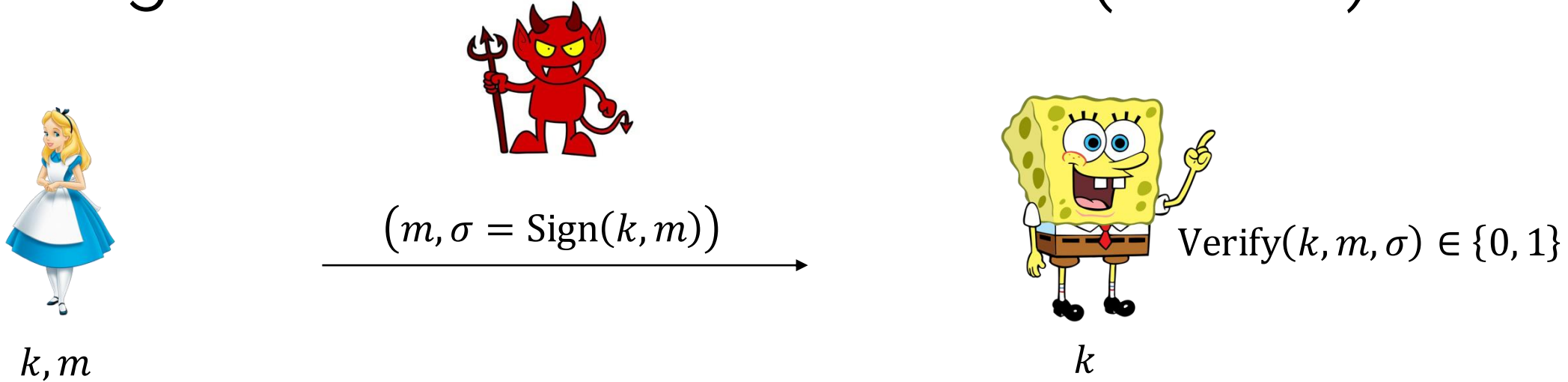
Defining Security for MACs



Win if:

- $\text{Verify}(k, m^*, \sigma^*) = 1$, and
- m^* was not queried.

Message Authentication Codes (MACs)



Definition: A **message authentication code (MAC)** scheme with message length $n = n(\lambda)$ consists of efficient algorithms **Sign**, **Verify**. It must satisfy the following properties.

- **Correctness:** for all k, m , $\text{Verify}(k, m, \text{Sign}(k, m)) = 1$ with probability 1.
- **Unforgeability:** for all efficient oracle adversaries $A^{\text{Sign}(\textcolor{red}{k}, \cdot)}$, we have:

$$\Pr[A^{\text{Sign}(\textcolor{red}{k}, \cdot)}(1^\lambda) \rightarrow (m^*, \sigma^*): m^* \text{ was not queried and } \text{Verify}(\textcolor{red}{k}, m^*, \sigma^*) = 1] = \text{negl}(\lambda)]$$